	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorio de docencia

Laboratorio de Computación Salas A y B

Profesor(a): M.C. René Adrián Dávila Pérez

Asignatura: Programación orientada a objetos

Grupo: 07

No de práctica(s): 07

Integrante(s): 424045919 | 323087067 | 426060033 | 426059961 | 323297930

No de lista o brigada: 01

Semestre: 2027-1

Fecha de entrega: 16 de octubre de 2026

Observaciones:

Calificación:

Índice

1. Introducción **2**

1.1. Planteamiento del Problema 2

1.2. Motivación 2

2. Objetivos **2**

2.1. Objetivo General 2

2.2. Objetivos Específicos 2

3. Marco Teórico **3**

3.1. Herencia y Extensión de Funcionalidades en POO 3

3.2. Relaciones de Composición (HAS-A) e Identidad (IS-A) 3

3.3. Polimorfismo, Sobrescritura de Métodos y Referencias `super` 3

4. Desarrollo **4**

4.1. Actividad 1: Consulta Técnica con IA sobre el Encapsulamiento `protected` en Herencia 4

4.2. Actividad 2: Implementación de la Jerarquía de Cuadriláteros por Herencia Profunda 4

4.3. Diagramas de Clases UML 5

4.4. Casos de Prueba y Metodología de Validación 6

5. Resultados **7**

5.1. Consultas a IAs por la Brigada 7

5.2. Evidencias de Ejecución en Terminal 8

6. Conclusiones **8**

7. Referencias **9**

1. Introducción

1.1. Planteamiento del Problema

Cuando desarrollamos software orientado a objetos y necesitamos modelar entidades que comparten características similares, programar cada clase desde cero genera repetición innecesaria de código. Por ejemplo, al trabajar con figuras geométricas de cuatro lados, escribir de forma independiente la lógica para almacenar vértices o calcular áreas en cada tipo de figura complica el mantenimiento del sistema. En esta práctica resolvemos este problema construyendo una jerarquía de herencia profunda en Java para la familia de los cuadriláteros: la clase base `Cuadrilatero`, de la cual se derivan en cascada `Trapezoide`, `Paralelogramo`, `Rectangulo` y `Cuadrado`. Además, integramos una clase `Punto` para gestionar las coordenadas en el plano mediante composición y aplicamos la fórmula general del trapecio para resolver el cálculo de área en todas las subclases de forma heredada.

1.2. Motivación

La herencia es un concepto clave en la POO porque nos permite construir clases nuevas a partir de clases ya existentes, aprovechando sus atributos y métodos sin tener que reescribirlos. Diseñar una estructura jerárquica clara del tipo IS-A simplifica el código, fomenta la reutilización algorítmica y prepara el programa para aplicar polimorfismo. Del mismo modo, analizar el uso del modificador `protected` frente a campos privados nos ayuda a tomar decisiones de diseño más seguras y ordenadas en nuestros proyectos.

2. Objetivos

2.1. Objetivo General

Implementar los conceptos de herencia en un lenguaje de programación orientado a objetos.

2.2. Objetivos Específicos

- Construir una jerarquía de herencia profunda que enlace en cascada las clases `Cuadrilatero`, `Trapezoide`, `Paralelogramo`, `Rectangulo` y `Cuadrado`.
- Aplicar el concepto de composición mediante la clase `Punto` para almacenar las coordenadas cartesianas de los vértices de cada figura.
- Definir el cálculo del área en la clase `Trapezoide` mediante la fórmula general del trapecio, permitiendo que las clases hijas lo reutilicen directamente.
- Analizar mediante consultas con distintas herramientas de IA los inconvenientes de utilizar visibilidad `protected` en herencia.
- Validar el comportamiento del sistema instanciando los objetos en una clase de prueba e imprimiendo el área calculada para cada figura.

3. Marco Teórico

3.1. Herencia y Extensión de Funcionalidades en POO

La herencia es el mecanismo mediante el cual una clase nueva absorbe los atributos y métodos de una clase base existente. En Java, todas las clases heredan implícitamente de la superclase `Object`, y para extender las funcionalidades de una clase personalizada se utiliza la palabra reservada `extends`. La clase origen se conoce como superclase o clase base, mientras que la clase derivada recibe el nombre de subclase o clase hija. Esta relación promueve la reutilización de código y constituye la base para el polimorfismo [1, 3].

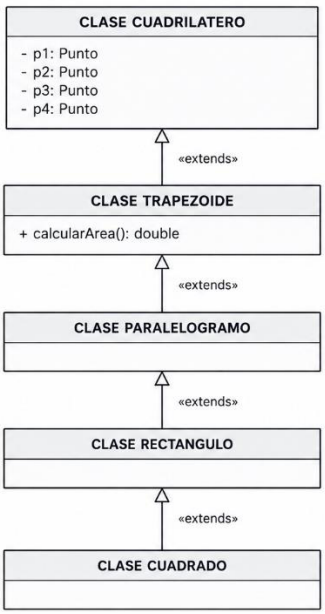


Figura 1: Diagrama UML de la jerarquía de herencia en cascada para la familia de cuadriláteros.

3.2. Relaciones de Composición (HAS-A) e Identidad (IS-A)

Al diseñar un sistema orientado a objetos es importante distinguir entre dos tipos de relaciones [1]:

- **Relación IS-A:** Se establece mediante herencia y define una jerarquía categórica [1]. Por ejemplo, un `Cuadrado` es un `Rectangulo`, y un `Rectangulo` es un `Paralelogramo`.
- **Relación HAS-A:** Se basa en la composición, donde una clase utiliza instancias de otra para construir su propio estado [1]. En nuestro modelo, un `Cuadrilatero` tiene cuatro objetos de tipo `Punto`.

3.3. Polimorfismo, Sobrescritura de Métodos y Referencias `super`

La sobrescritura de métodos permite a una subclase adaptar un comportamiento heredado, indicándolo explícitamente con la anotación `@Override` [1]. Por otro lado, la palabra reservada `super`

actúa como una referencia hacia la superclase inmediata, permitiendo invocar sus constructores o métodos desde la clase hija para asegurar que los atributos heredados se inicialicen correctamente [2].

4. Desarrollo

4.1. Actividad 1: Consulta Técnica con IA sobre el Encapsulamiento `protected` en Herencia

Para el primer ejercicio, cada integrante de la brigada consultó a una herramienta de IA diferente (ChatGPT, Claude, Gemini, Copilot y DeepSeek) sobre la siguiente pregunta: "¿Por qué no es buena práctica usar encapsulamiento protected en herencia?".

El análisis de las respuestas concluye que utilizar el modificador `protected` rompe parcialmente el encapsulamiento, ya que no solo expone las variables a las clases hijas, sino también a cualquier otra clase que se encuentre dentro del mismo paquete. Esto permite que componentes ajenos a la jerarquía modifiquen el estado interno de un objeto sin pasar por filtros de validación, aumentando el acoplamiento y complicando el mantenimiento del código. Por ello, la regla de diseño recomendada es mantener los atributos como privados (`private`) y brindar acceso únicamente mediante métodos de lectura y escritura (getters y setters).

4.2. Actividad 2: Implementación de la Jerarquía de Cuadriláteros por Herencia Profunda

Diseñamos e implementamos una estructura de clases modular para organizar las figuras geométricas de cuatro lados:

1. **Clase `Punto`:** Representa una posición en el plano cartesiano mediante dos variables privadas `x` e `y` de tipo decimal, junto con su constructor y métodos `getX()` y `getY()`.

2. **Clase `Cuadrilatero`:** Funciona como superclase para la jerarquía. Aplica composición incluyendo cuatro objetos `Punto` (`p1`, `p2`, `p3`, `p4`) como atributos privados y ofrece sus correspondientes métodos de acceso.

3. **Clase `Trapezoide`:** Hereda de `Cuadrilatero` invocando a su constructor mediante `super(p1, p2, p3, p4)`. Define el método `calcularArea()`, el cual obtiene la base mayor, base menor y altura a partir de la distancia entre coordenadas y aplica la fórmula general:

$$\frac{(\text{baseMayor} + \text{baseMenor}) \times \text{altura}}{2.0}.$$

4. **Clases `Paralelogramo`, `Rectangulo` y `Cuadrado`:**

   - `Paralelogramo` hereda de `Trapezoide`.
   - `Rectangulo` hereda de `Paralelogramo`.
   - `Cuadrado` hereda de `Rectangulo`.

- Gracias a esta herencia en cascada, ninguna de estas subclases requiere reescribir la lógica del cálculo de área, ya que reutilizan automáticamente la fórmula implementada en `Trapezoide`.

5. **Clase de Prueba:** Instancia los objetos para cada una de las figuras especificando sus coordenadas, ejecuta el método de cálculo e imprime el área resultante en consola.

4.3. Diagramas de Clases UML

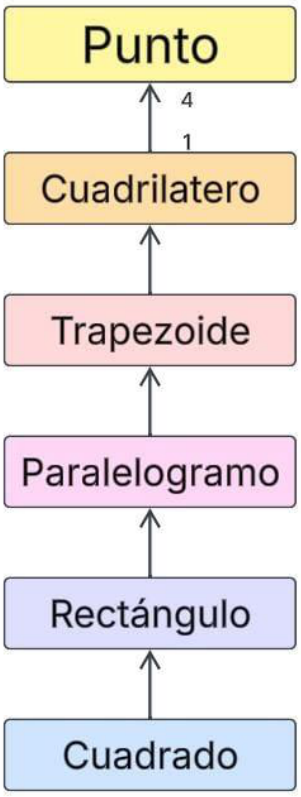


Figura 2: Esquema de la jerarquía de clases y la relación de composición de los cuadriláteros con la clase `Punto`.

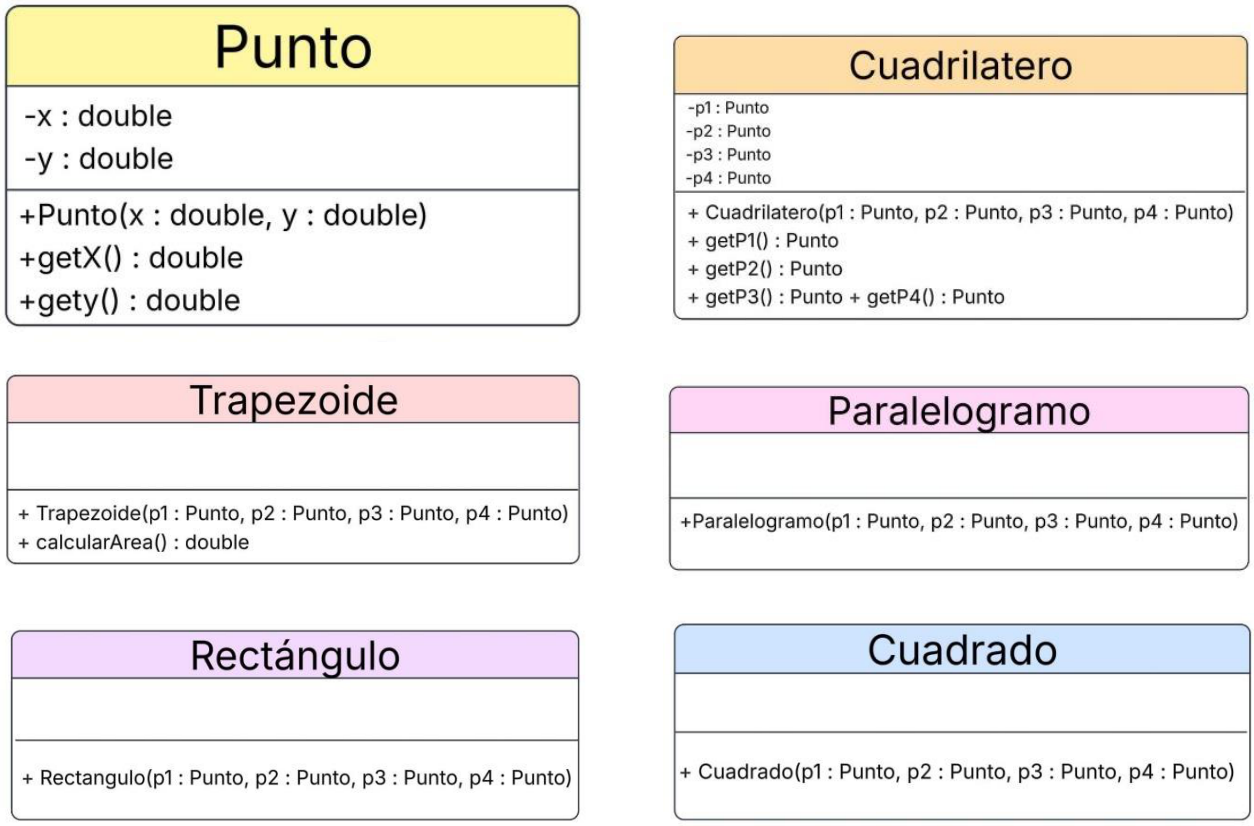


Figura 3: Diagramas UML detallados de los atributos y métodos de las clases de la jerarquía geométrica.

4.4. Casos de Prueba y Metodología de Validación

Metodología de Pruebas

Probamos el funcionamiento de la jerarquía ingresando vértices conocidos y evaluando la salida del área en cada nivel:

1. **Trapezoide:** Puntos con base mayor 8.0, base menor 4.0 y altura 4.0, obteniendo un área esperada de 24.0.
2. **Paralelogramo:** Puntos con bases iguales de 6.0 y altura 4.0, obteniendo un área calculada de 24.0.
3. **Rectángulo:** Puntos con base 6.0 y altura 4.0, dando un área de 24.0 mediante herencia en cascada.
4. **Cuadrado:** Puntos formando lados iguales de 5.0, comprobando que la fórmula heredada devuelve un área de 25.0.

Tabla 1: Pruebas de Escritorio

Figura geométrica	Coordenadas de vértices	Fórmula aplicada	Área esperada	Área obtenida	Estado
Trapezoide	$(0,0),(8,0),(6,4),(2,4)$	$\frac{(8.0+4.0)\times 4.0}{2.0}$	24.0	24.0	Éxito
Paralelogramo	$(0,0),(6,0),(8,4),(2,4)$	$\frac{(6.0+6.0)\times 4.0}{2.0}$	24.0	24.0	Éxito
Rectángulo	$(0,0),(6,0),(6,4),(0,4)$	$\frac{(6.0+6.0)\times 4.0}{2.0}$	24.0	24.0	Éxito
Cuadrado	$(0,0),(5,0),(5,5),(0,5)$	$\frac{(5.0+5.0)\times 5.0}{2.0}$	25.0	25.0	Éxito

5. Resultados

5.1. Consultas a IAs por la Brigada

A continuación mostramos las consultas realizadas por los miembros de la brigada para resolver el primer ejercicio:

[Integrante 1: Bazán Pantaleón Rosa María | Herramienta: ChatGPT (OpenAI)]
PROMPT: "¿Por qué no es buena práctica usar encapsulamiento protected en herencia? Justifica técnicamente."
RESPUESTA: "Usar protected debilita el encapsulamiento porque permite que cualquier clase en el mismo paquete acceda a los atributos sin pasar por validaciones. En jerarquías grandes, esto genera un acoplamiento innecesario y hace que los cambios en la superclase afecten a otras clases."

[Integrante 2: García Rivera Gabriel Eduardo | Herramienta: Copilot (Microsoft)]
PROMPT: "Explica por qué los expertos en Java recomiendan evitar el modificador protected en atributos de superclases."
RESPUESTA: "Porque incrementa el acoplamiento y expone la estructura interna de la clase. Aunque facilita la lectura directa en subclases, abre un acceso no deseado a nivel de paquete."

[Integrante 3: Padrón López María Fernanda Corina | Herramienta: Gemini (Google)]
PROMPT: "¿Cuáles son los riesgos de seguridad y diseño al usar protected en lugar de private en POO Java?"
RESPUESTA: "El riesgo es perder el control sobre cómo se modifican los datos. Las variables protected se pueden alterar desde clases vecinas del paquete. Mantener atributos privados con getters y setters asegura la integridad de la información."

[Integrante 4: Toledo Valencia Aimee Guadalupe | Herramienta: Claude (Anthropic)]
PROMPT: "Analiza las desventajas arquitectónicas de emplear modificadores protected en clases base con herencia en Java."
RESPUESTA: "La desventaja principal es que rompe el principio de ocultamiento de datos. Al marcar una variable como protected, no solo es accesible por las clases hijas, sino por todo el paquete. Lo recomendado es usar private y métodos de acceso."

[Integrante 5: Vitla Herrera Cesar Antonio | Herramienta: DeepSeek]
PROMPT: "Justifica por qué el uso de protected en jerarquías de clases se considera un anti-patrón de diseño."
RESPUESTA: "Se considera una mala práctica porque combina el acceso por herencia con el acceso por paquete. Esto permite que clases no relacionadas modifiquen variables internas si están en la misma carpeta, restando modularidad."

5.2. Evidencias de Ejecución en Terminal

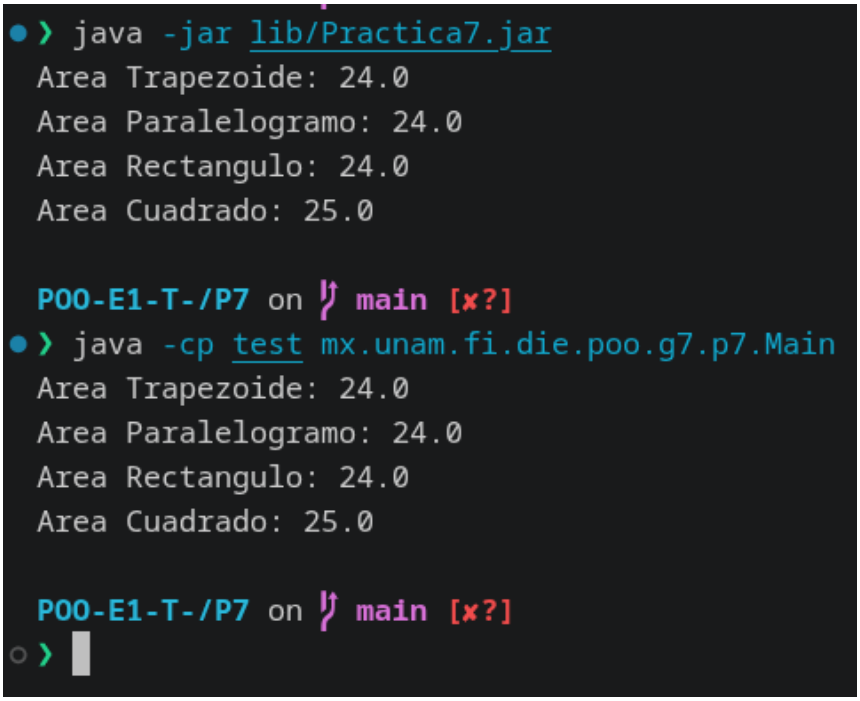


Figura 4: Impresión en terminal con el cálculo de áreas reutilizado mediante herencia.

6. Conclusiones

La realización de esta práctica nos permitió verificar la utilidad de la herencia y la composición para modelar jerarquías de clases en la programación orientada a objetos. Al integrar la clase `Punto` dentro de `Cuadrilatero`, aplicamos la composición (HAS-A) para almacenar de forma segura las coordenadas de los vértices de las figuras. Por otro lado, la construcción en cascada de las subclases `Trapezoide`, `Paralelogramo`, `Rectangulo` y `Cuadrado` demostró el beneficio de las relaciones de herencia (IS-A), ya que al centralizar el método de cálculo de área en la clase base trapecio, todas las figuras derivadas pudieron calcular su área correctamente mediante `super` sin duplicar líneas de código. Finalmente, el análisis de la visibilidad `protected` reforzó la importancia del encapsulamiento estricto, confirmando que proteger los atributos como privados y acceder a ellos mediante métodos públicos mantiene la estabilidad y el orden en nuestros programas.

7. Referencias

[1] Universidad Nacional Autónoma de México (UNAM CUAED), "Herencia," Repositorio de la Coordinación de Universidad Abierta y Educación a Distancia, [En línea]. Disponible en: https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3057/mod_resource/content/1/UAPA-Herencia/index.html

[2] W3Schools, "Java super Keyword," W3Schools Online Web Tutorials, [En línea]. Disponible en: https://www.w3schools.com/java/java_super.asp

[3] W3Schools, "Java Inheritance (Subclass and Superclass)," W3Schools Online Web Tutorials, [En línea]. Disponible en: https://www.w3schools.com/java/java_inheritance.asp